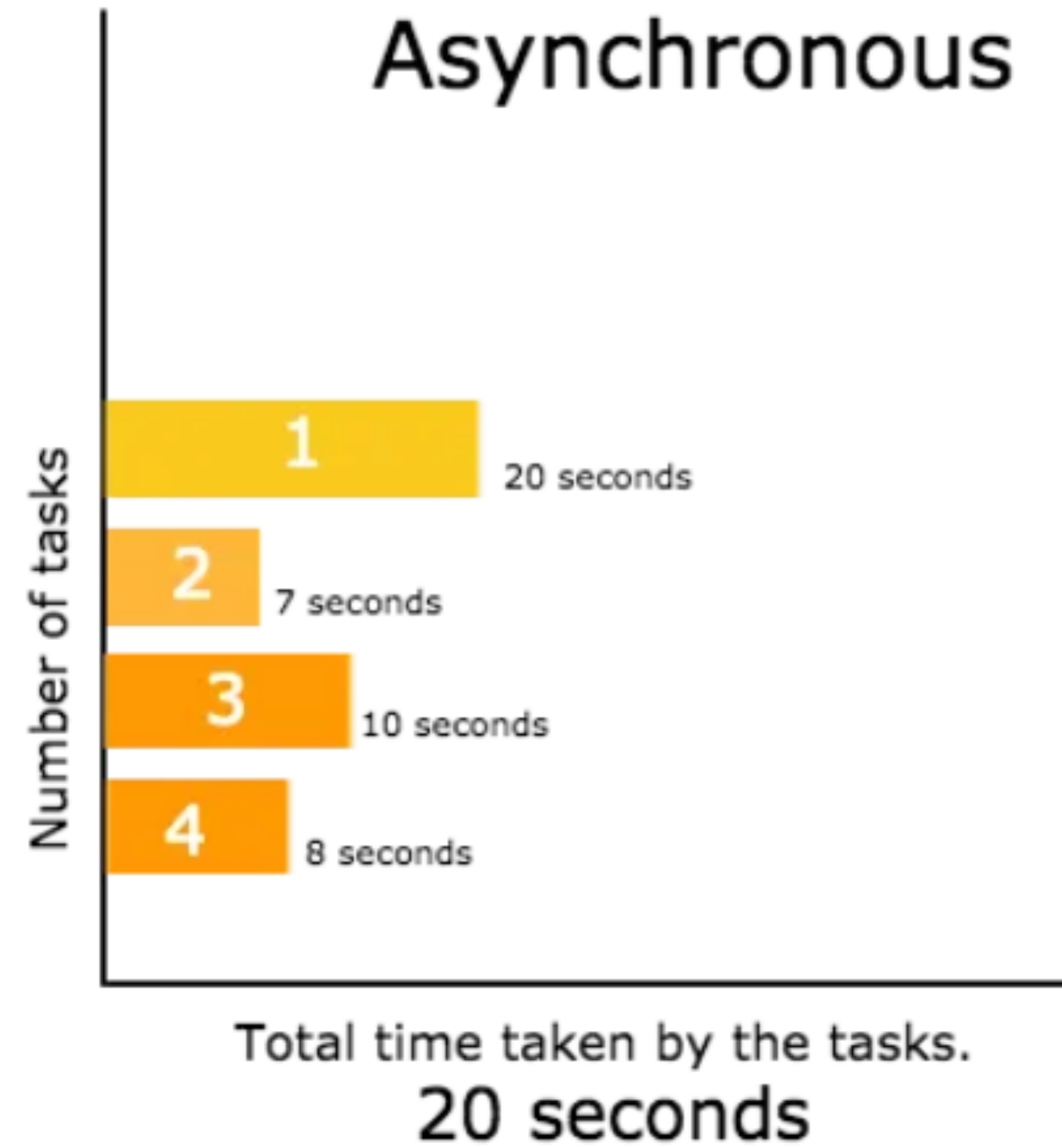
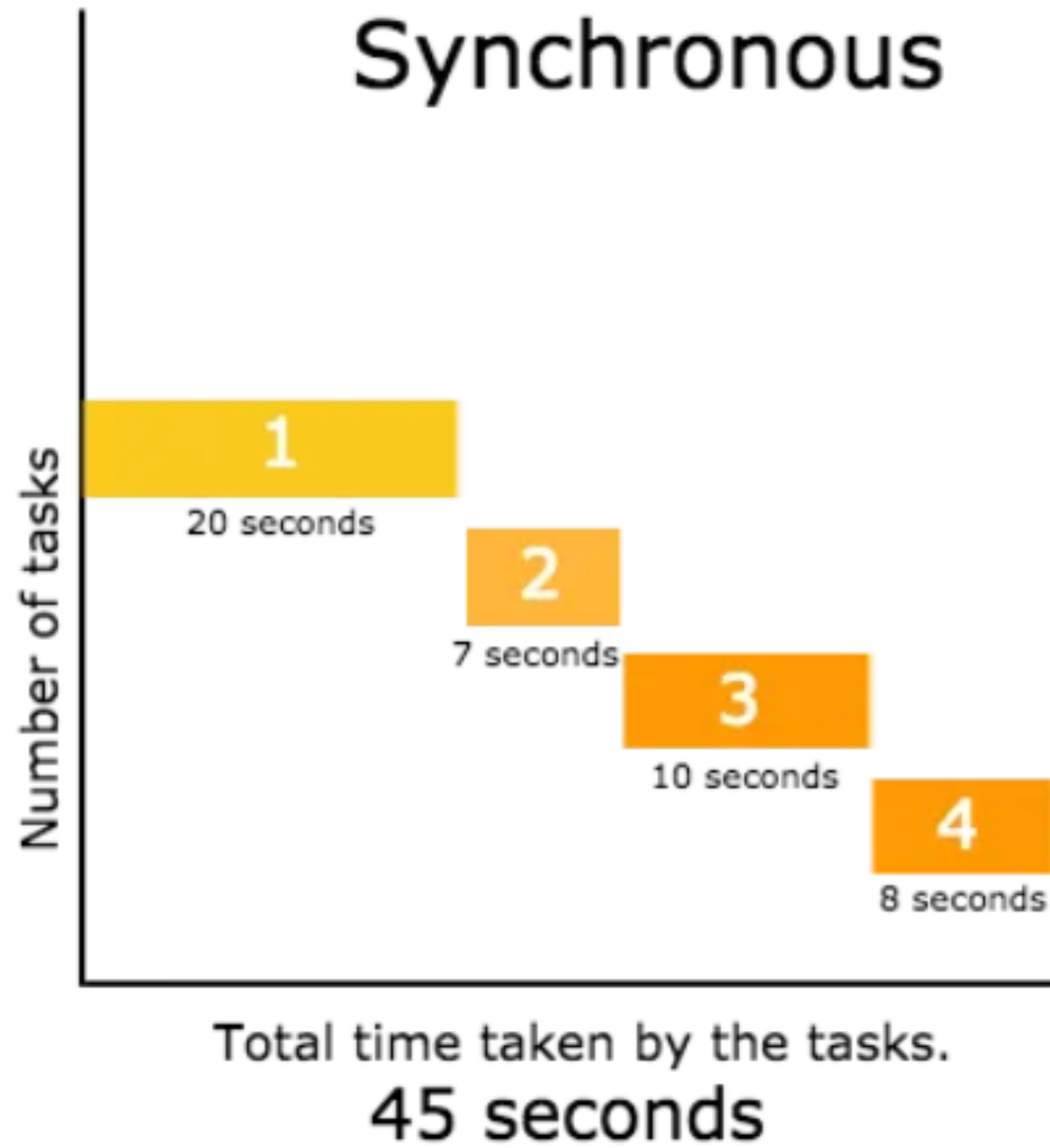


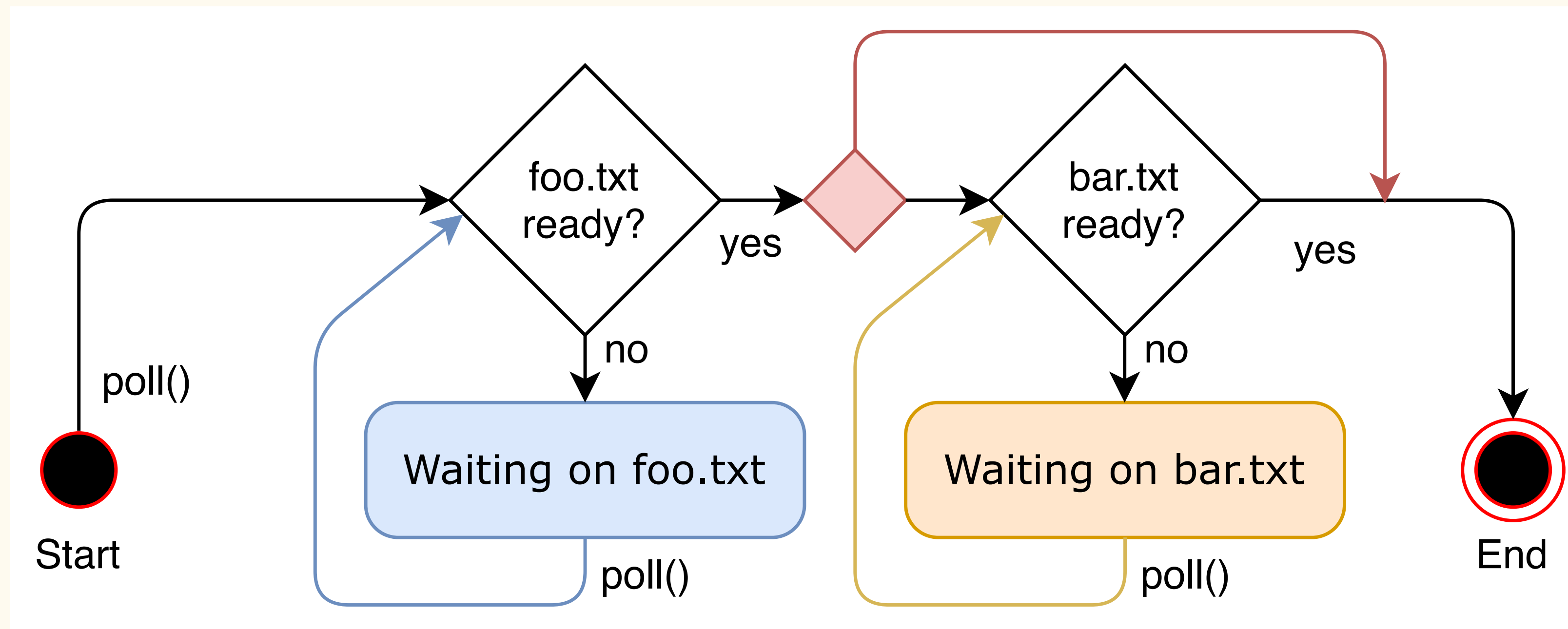
ASYNC RUST



I. Lesson



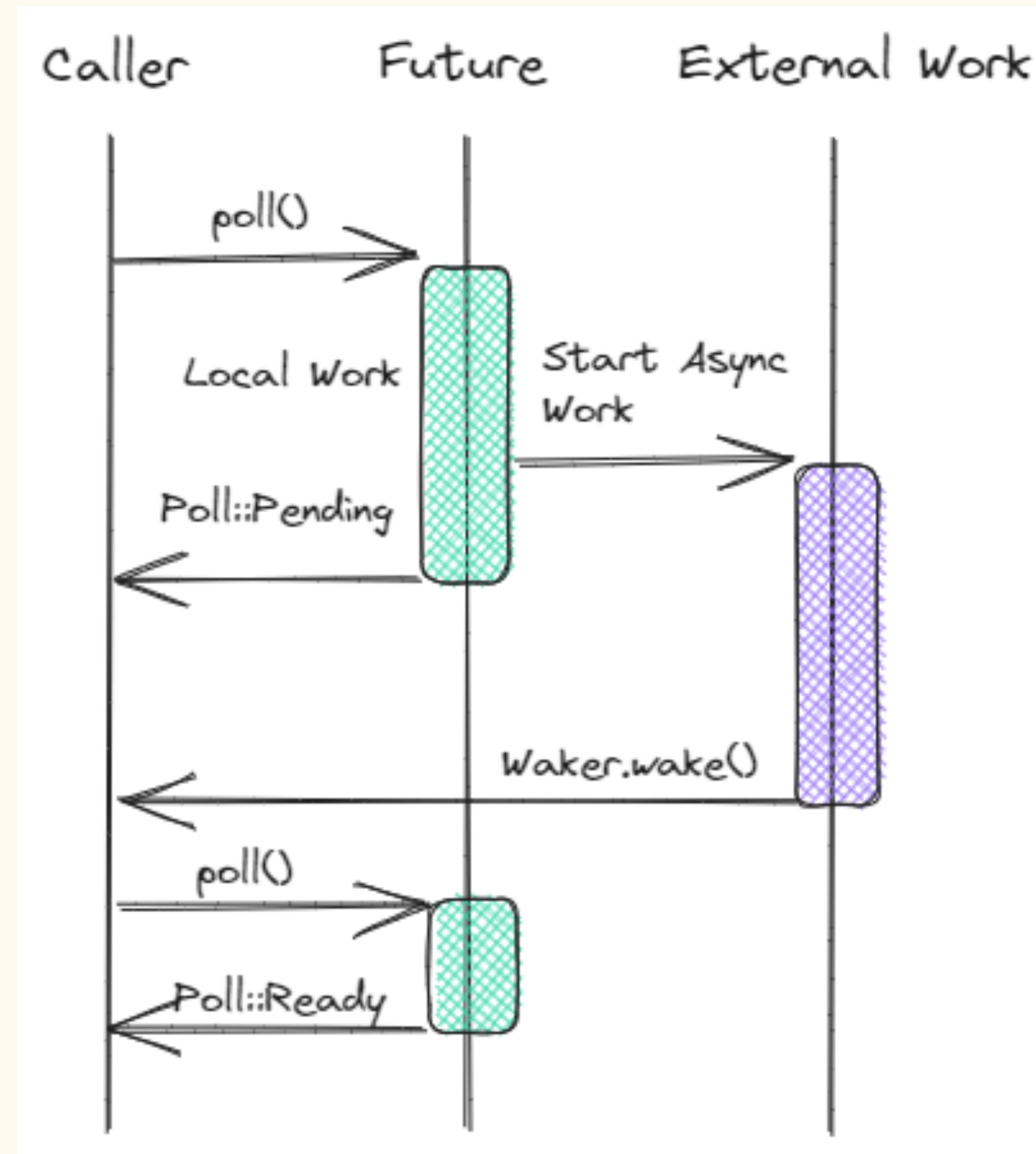
Understanding poll()





```
pub enum Poll<T> {  
    Ready(T),  
    Pending,  
}
```


Understanding the Waker



Pinning

- **Pinning** — This is a mechanism for attaching data to a specific location in memory. By using **Pin**, you guarantee that the object won't be moved to another location in memory. For example, **`Pin<&mut Self>`** indicates that `&mut Self` will always be in the same memory location between the calls to **`Self::poll`**. Using the same memory location is mandatory if you want to be sure that the fields of **`Self`** don't change their memory address.

II. Project

Sync vs Async



```
// Regular function - executes immediately
fn regular_function() -> i32 {
    println!("Regular function executing NOW");
    42
}

// Async function - returns a Future (lazy,
// doesn't execute yet)
async fn async_function() -> i32 {
    println!("Async function executing NOW");
    42
}
```

Blocking & Non-blocking

```
#[tokio::main]
async fn main() {
    println!("=== BLOCKING CODE ===");
    let start = std::time::Instant::now();

    blocking_task(1);
    blocking_task(2);
    blocking_task(3);

    println!("Total time: {:?}\n", start.elapsed());
    // Each task waits, one after another = 6 seconds total

    println!("=== NON-BLOCKING CODE ===");
    let start = std::time::Instant::now();

    // Run all tasks concurrently
    tokio::join!(
        non_blocking_task(1),
        non_blocking_task(2),
        non_blocking_task(3)
    );

    println!("Total time: {:?}", start.elapsed());
    // All tasks run at the same time = only 2 seconds total!
}
```


Cancelling

```
● ● ●

#[tokio::main]
async fn main() {
    println!("=== Without Cancellation ===");
    long_running_task(1).await;
    println!();

    println!("=== With Cancellation ===");

    // Create a task handle
    let task = tokio::spawn(long_running_task(2));

    // Let it run for a bit
    sleep(Duration::from_secs(2)).await;

    // Cancel the task by dropping it
    println!("Cancelling task 2...");
    task.abort();

    // Check if cancelled
    match task.await {
        Ok(_) => println!("Task completed normally"),
        Err(e) if e.is_cancelled() => println!("Task was cancelled!"),
        Err(e) => println!("Task failed: {}", e),
    }
}
```

Poll mechanism & Waker

```
● ● ●  
  
// Custom Future that counts to 3  
struct CounterFuture {  
    count: u32,  
}  
  
impl Future for CounterFuture {  
    type Output = u32;  
  
    fn poll(mut self: Pin<&mut Self>, cx: &mut Context) ->  
    Poll<Self::Output> {  
        self.count += 1;  
        println!("Polling... count is now {}", self.count);  
  
        if self.count < 3 {  
            // Not ready yet - wake ourselves to be polled again  
            cx.waker().wake_by_ref();  
            Poll::Pending  
        } else {  
            // Ready!  
            Poll::Ready(self.count)  
        }  
    }  
}
```


Let's code!

Your turn!